

DESIGN CHOICES DOCUMENT

Real-Time Financial Fraud Detection System (Version 1 to Version 2 Architectural Redesign)

Summary

This document explains the architectural redesign of my real-time fraud detection system from Version 1 to Version 2. The redesign addresses five critical limitations identified during V1 operation: data quality for ML training, system scalability, state consistency, production readiness, and type safety.

Business Context: V1 successfully demonstrated end-to-end ML inference with PyFlink and automated retraining via Airflow. V2 redesigns the data generation and stream processing layers to handle production-scale traffic while maintaining data quality suitable for ML model training.

Key Metrics:

- Sustained throughput increased from 1.8 TPS to 6,900 TPS
- Peak capacity: 9,150 TPS
- P99 latency: 16.9ms under sustained load
- Reliability: 99.99% success rate over 414K requests

Scope: V2 focuses exclusively on ingestion and enrichment. ML inference components from V1 will be migrated in V3.

Version 1 Overview

What V1 Accomplished

V1 demonstrated a complete fraud detection pipeline including data generation, stream enrichment, real-time ML inference, and continuous training automation.

Architecture Summary:

- Go-based transaction simulator feeding Kafka

- Stream enrichment adding GeoIP data
- PyFlink consuming enriched streams for real-time ML inference
- Airflow orchestrating model retraining with MLflow tracking and DVC versioning

V1 Strengths:

- End-to-end ML pipeline operational
- Hot-reload model serving functional
- Automated retraining demonstrated
- Complete MLOps workflow

V1 Throughput: 1.8 TPS (intentionally limited for demonstration purposes)

Identified Limitations Requiring Redesign

Limitation 1: Statistically Unrealistic Synthetic Data

Problem: V1 generated random transactions using Go's math/rand with uniform distributions. Real financial data follows power-law distributions with complex feature correlations.

Impact: ML models trained on V1 data showed poor performance on real-world fraud patterns. The data didn't capture statistical properties of actual financial transactions.

Root Cause: Random number generation cannot reproduce the joint probability distributions found in real datasets.

Business Consequence: Cannot validate ML model accuracy or iterate on feature engineering with confidence.

Limitation 2: No Behavioral User Modeling

Problem: Transactions had no user context. Same user could appear from completely different countries with no travel time, violating basic behavioral patterns.

Impact: Missing critical fraud signals like sudden location changes, VPN usage patterns, or device switching that real fraud detection relies on.

Root Cause: No state maintained between transactions for individual users.

Limitation 3: Schema Fragility

Problem: JSON serialization with manual field mapping between Go and Python services. Field name mismatches like "oldbalanceOrg" vs "oldbalanceOrig" caused silent data loss.

Impact: Production deployment risk due to runtime-only error detection. No compile-time validation of schema compatibility.

Root Cause: No shared schema definition between producer and consumer services.

Limitation 4: Scalability Bottleneck (Single Redis)

Problem: Single Redis instance limited both availability and horizontal scaling.

Impact:

- Entire cache layer lost on node failure
- Memory constrained to single machine
- Cannot distribute load across nodes

Root Cause: V1 design assumed single-node deployment.

Limitation 5: Race Conditions in State Updates

Problem: Fraud metrics computed using sequential Redis operations (GET, modify in application, SET). Concurrent transactions caused lost updates.

Impact: Fraud velocity calculations were inconsistent under load. Same IP could have different metric values depending on operation timing.

Root Cause: No atomic operation support for multi-field updates.

Limitation 6: No Rate Limiting or Deduplication

Problem: System had no protection against abuse, duplicate processing, or retry storms.

Impact:

- Single user could exhaust system resources
- Network retries caused duplicate fraud alerts
- No backpressure mechanism

Root Cause: Not implemented in V1 scope (focused on ML pipeline).

Design Approach: Why Complete Redesign?

Considered Approaches

Option A: Incremental Enhancement of V1

- Add Protocol Buffers alongside JSON
- Add Lua scripts to existing Redis operations
- Add gRPC endpoint while maintaining Kafka path
- Estimated effort: 2-3 weeks

Option B: Complete Architectural Redesign (Selected)

- Rebuild ingestion layer with Go microservices
- Adopt gRPC as primary communication protocol
- Redesign around Redis Cluster from ground up
- Estimated effort: 3-4 weeks

Why Option B Was Selected

Technical Reasons:

1. **Language Separation of Concerns:** V1's Python monolith couldn't efficiently handle both statistical data generation (SDV/Mesa) and high-throughput network I/O. Splitting into Python (data science) and Go (infrastructure) allows each service to operate in its strength domain.
2. **Schema Migration Complexity:** Converting V1's JSON to Protocol Buffers requires coordinated changes across all services. Incremental migration would

leave the system in a broken intermediate state. Clean redesign avoids compatibility layers.

3. **Redis Cluster Hash Tag Requirements:** Redis Cluster requires key naming patterns (hash tags) that are incompatible with V1's key structure. Every Redis operation needed rewriting, not just enhancement.
4. **Atomic Operations Requirement:** Adding Lua scripts to V1's sequential operation logic would require complete refactoring of state management anyway. Starting fresh allowed designing for atomicity from the beginning.

Accepted Tradeoff: V2 temporarily loses ML inference and MLOps features from V1. These will be reintegrated in V3 after validating the new architecture's performance characteristics. Attempting to maintain both simultaneously would significantly slow development.

Version 2 Design Decisions

Design Decision 1: Mesa + SDV for Synthetic Data Generation

Objective: Generate statistically realistic financial transactions suitable for ML model training.

Problem to Solve: V1's uniform random distributions don't match real financial data. Need both statistical accuracy and behavioral patterns.

Solution: Hybrid approach combining statistical modeling (SDV) with behavioral simulation (Mesa).

Component 1 - SDV Gaussian Copula:

- Pre-trained on 2M transactions from Databricks FSI fraud dataset using PySpark
- Preserves marginal distributions and feature correlations
- Conditional sampling based on fraud label
- Generates: amounts, types, balance changes, overdraft flags

Why Gaussian Copula?

The following comparison reflects theoretical and literature-based expectations, not empirical benchmarking within this project.

Method	Inference Speed	Data Quality	Decision
GAN	Fast	Excellent	Rejected (GPU requirement)
CTGAN	Slow	Excellent	Rejected (computational cost)
Gaussian Copula	Fast	Good	Selected
Simple sampling	Fastest	Poor	V1 baseline

Gaussian Copula selected for CPU-only training/inference while preserving statistical properties needed for ML validation.

Component 2 - Mesa Agent-Based Modeling:

- Each user represented as independent agent
- Agents maintain persistent state: home IP, domestic travel pool
- Context-aware IP generation based on fraud probability
- Legitimate users: 85% home, 10% domestic, 5% foreign
- Fraudulent users: 80% foreign, 20% home spoofing

Why Mesa for Behavioral Patterns?

Pure SDV cannot generate IP addresses with behavioral consistency (same user from same location). Mesa agents maintain user state across transactions, creating realistic geographic patterns critical for fraud detection.

Limitation Acknowledged: Mesa is designed for multi-agent simulations, not data generation. Simpler class-based approach could achieve similar results with less complexity. Selected to maintain extensibility for future fraud network simulations.

Outcome: Can now generate unlimited statistically realistic transactions matching production fraud patterns.

Design Decision 2: Protocol Buffers + gRPC

Objective: Eliminate schema-related runtime errors and enable type-safe cross-language communication.

Problem to Solve: V1's JSON serialization caused silent data loss from field name typos. No compile-time validation of schema compatibility between Go and Python services.

Solution: Protocol Buffers as single source of truth with gRPC for service communication.

Key Benefits:

Requirement	JSON (V1)	Protocol Buffers (V2)
Type Safety	Runtime only	Compile-time
Cross-Language	Manual mapping	Auto-generated stubs
Schema Evolution	Breaking changes	Backward compatible via field numbers
Serialization	Text-based	Binary

Why gRPC Over REST?

Factor	REST + JSON	gRPC + Protobuf	Rationale
Type Safety	None	Strong	Catch errors at build time
Performance	Baseline	3-5x faster	Industry benchmark
Cross-Language	Manual	Generated	Reduce integration errors
Streaming	Not native	Bidirectional	Future extensibility

Decision: Selected gRPC for type safety and performance. Accepts additional complexity of protobuf toolchain for production-grade schema management.

Outcome: Zero schema-related runtime errors during testing. All field mismatches caught at compile time.

Design Decision 3: Redis Cluster Over Single Node

Objective: Eliminate single point of failure and enable horizontal scaling of cache layer.

Problem to Solve: V1's single Redis node created availability risk and memory constraints.

Solution: 3-node Redis Cluster with hash-based sharding.

Architecture: Three master nodes without replicas, distributing 16,384 hash slots across nodes.

Why 3 Masters Without Replicas?

Evaluated configurations:

Configuration	Availability	Complexity	Resource Cost	Decision
1 node (V1)	Poor (SPOF)	Simple	1x	Baseline
3 masters, 0 replicas	Manual failover	Medium	3x	V2 (development)
3 masters, 3 replicas	Automatic failover	Higher	6x	V3 (production)

Selected 3 masters for:

- Horizontal memory scaling (3x capacity)
- Learning cluster operations
- Sharding for load distribution
- Development environment appropriateness

Acknowledged Limitation: No automatic failover. Node failure requires manual intervention. Production deployment requires replicas.

Hash Tag Design: Keys use `{ip}` pattern to ensure related keys (geo, fraud metrics, dedup) map to same hash slot, enabling atomic multi-key operations in Lua scripts.

Outcome: Distributed cache with 3x memory capacity and no single point of failure for data loss.

Design Decision 4: Atomic Lua Scripts for State Management

Objective: Eliminate race conditions in fraud metric calculations.

Problem to Solve: V1 pattern of GET-modify-SET allowed concurrent transactions to cause lost updates.

Solution: Server-side Lua script execution for atomic multi-operation updates.

What Changed:

Approach	Operations	Network Calls	Consistency
Sequential (V1 pattern)	7 Redis operations	7 network round trips	Race conditions possible
Atomic Lua (V2)	7 Redis operations	1 network round trip	Guaranteed atomic

Operations Combined in Single Lua Script:

1. Check if transaction already processed (idempotency)
2. Mark transaction as processed
3. Retrieve existing fraud data
4. Update transaction count
5. Update total amount
6. Recalculate velocity and averages
7. Update maximum amount

Network Efficiency: Eliminates 6 network round trips per transaction.

Outcome: Guaranteed consistency in fraud metrics under concurrent load. Measured P99 latency of 16.9ms confirms low overhead.

Design Decision 5: Distributed Token Bucket Rate Limiting

Objective: Protect system from abuse and ensure fair resource allocation.

Problem to Solve: V1 had no rate limiting. Single user could overwhelm all downstream services.

Solution: Distributed token bucket algorithm implemented via Redis Lua script.

Why Token Bucket?

Algorithm	Allows Bursts	Fairness	Complexity	Decision
Fixed Window	No	Poor	Low	Rejected
Sliding Window	No	Excellent	High	Rejected
Token Bucket	Yes	Good	Medium	Selected
Leaky Bucket	No	Good	Medium	Rejected

Selected token bucket because:

- Legitimate users make burst purchases (multiple items in cart)
- Prevents sustained abuse while allowing temporary spikes
- Industry standard (AWS, Stripe, GitHub use token bucket)

Configuration:

- Burst capacity: 1,000 tokens per user
- Refill rate: 500 tokens/second per user
- Distributed enforcement via Redis (works across multiple server instances)

Why Redis-Based vs In-Memory?

With multiple go-server instances, in-memory rate limiting would allow N instances $\times$ 500 = $N \times 500$ req/sec per user. Redis enforces global limit across all instances.

Outcome: Successfully rate-limited during load testing. System rejected requests exceeding limits while maintaining sub-20ms latency for accepted requests.

Design Decision 6: Multi-Layer Deduplication

Objective: Guarantee exactly-once processing despite network retries and at-least-once Kafka delivery.

Problem to Solve: Network failures cause client retries. Kafka provides at-least-once delivery guarantees. Both create duplicate processing risk.

Solution: Deduplication at two layers with different time windows.

Layer 1 - Ingestion Server:

- Deduplication window: 10 minutes
- Purpose: Protect against immediate client retries
- Implementation: Redis SetNX operation

Layer 2 - Enrichment Service:

- Deduplication window: 2 hours
- Purpose: Prevent duplicate fraud metric updates

- Implementation: Set membership check in Lua script (atomic with metric update)

Why Different Windows?

Layer	Window	Rationale
Ingestion	10 minutes	Client retries occur within seconds/minutes
Enrichment	2 hours	Matches fraud velocity tracking period

Known Issue: Deduplication key rollback on rate limit failure is non-atomic. If rollback fails, future legitimate retries will be incorrectly rejected. Future enhancement will combine rate limiting and deduplication in single Lua script.

Outcome: Zero duplicate fraud metrics observed during 414K request test.

Design Decision 7: Manual Kafka Offset Management

Objective: Achieve exactly-once processing semantics.

Problem to Solve: Kafka auto-commit can acknowledge messages before processing completes, causing data loss on consumer failure.

Solution: Manual offset commits with dual-trigger strategy.

Commit Triggers:

1. Batch-based: Every 20 successfully processed messages
2. Time-based: Every 5 seconds if any pending messages
3. Shutdown: Final commit on SIGINT/SIGTERM before exit

Why Dual Strategy?

Trigger	Purpose
Batch (20 messages)	Efficient under high throughput
Time (5 seconds)	Ensure progress during low traffic periods
Shutdown	Prevent message loss on graceful termination

Exactly-Once Guarantee: Only commit offset after successful publish to enriched Kafka topic. On failure, message reprocessed on consumer restart. Idempotency layer prevents duplicate fraud metrics.

Outcome: No message loss observed. Graceful shutdown tested successfully.

Design Decision 8: Enhanced GeoIP Enrichment

Objective: Capture fraud signals beyond basic location (city/country).

Problem to Solve: V1 only captured city and country. Fraud often originates from VPNs, hosting providers, or specific ASNs.

Solution: Dual MaxMind database approach extracting geographic and network infrastructure data.

Data Added in V2:

- Autonomous System Number (ASN)
- ISP/Organization name
- Hosting provider detection (VPN/cloud/datacenter identification)
- Latitude/Longitude for distance calculations

Why Hosting Provider Detection Matters:

- Fraudsters use VPNs and cloud providers to mask location
- Legitimate users rarely transact from AWS/GCP/Azure IPs
- Strong fraud indicator when combined with other signals

Caching Strategy:

- 24-hour TTL for GeoIP lookups (configurable)
- Look-aside cache pattern (check cache, fallback to database)
- Shared across all enricher instances via Redis Cluster

Outcome: 8 additional enrichment fields per transaction enabling richer fraud detection features for ML models.

Performance Validation

Load Testing Methodology

Tool: ghz (gRPC load testing framework)

Test Environment: Docker Compose on single host (all services local)

Payload Characteristics:

- Randomized user IDs (1000-9999 range)
- Random transaction amounts (\$10-\$100,000)
- Random IP addresses (full IPv4 space)
- Unique transaction IDs per request

Test Suite:

- 1K requests (20 concurrent connections)
- 10K requests (50 concurrent)
- 50K requests (100 concurrent)
- 100K requests (100 concurrent)
- 60-second sustained load (50 concurrent)

Results

Test Scenario	Requests	Throughput	P99 Latency	Success Rate
Light (1K)	1,000	2,820 TPS	58.67ms	100.00%
Medium (10K)	10,000	6,034 TPS	36.44ms	100.00%
Heavy (50K)	50,000	9,152 TPS	20.81ms	100.00%
Peak (100K)	100,000	6,863 TPS	34.06ms	100.00%
Sustained (60s)	414,163	6,903 TPS	16.92ms	99.99%

Performance Analysis

Sustained Performance: 6,903 TPS represents realistic production capacity. Peak of 9,152 TPS demonstrates headroom for traffic spikes.

Latency: P99 of 16.92ms means 99% of transactions complete in under 17ms. This meets real-time processing requirements for financial systems (typical target: <100ms).

Reliability: 99.99% success rate (41 failures in 414K requests). Failures attributed to transient Redis connection issues and Kafka buffer saturation during peak

burst.

Scalability Projection: Services are stateless. Horizontal scaling expected to be linear. Projected capacity:

- 2 instances: ~13,800 TPS
- 5 instances: ~34,500 TPS
- 10 instances: ~69,000 TPS

Technology Stack Comparison

Layer	V1	V2	Rationale for Change
Data Generation	Go (math/rand)	Python (Mesa + SDV)	Statistical realism
Communication	N/A	gRPC + Protobuf	Type safety
Ingestion	Go + Kafka	Go + gRPC + Kafka	Microservices pattern
Caching	Redis (single)	Redis Cluster (3 nodes)	Scalability
Enrichment	Go + MaxMind	Go + MaxMind + ASN	Richer fraud signals
Schema	JSON	Protocol Buffers	Strong typing
ML Inference	PyFlink + RF	Deferred to V3	Validate architecture first
MLOps	Airflow + MLflow + DVC	Deferred to V3	Reintegrate after validation

Known Limitations

Development vs Production Configuration

Current State (Development):

- Single Kafka broker (no replication)
- Redis Cluster without replicas (no automatic failover)
- No monitoring or observability stack

- No authentication on any service
- No TLS/encryption

Production Requirements (to be addressed in V3):

- Minimum 3 Kafka brokers with replication factor 3
- Redis replicas for automatic failover
- Prometheus metrics and Grafana dashboards
- mTLS for gRPC, passwords for Redis
- Distributed tracing for debugging

Code-Level Gaps

1. **Error Handling:** Kafka produce failures return success response to client (message loss risk)
2. **Context Timeouts:** No bounded timeouts on Redis/Kafka operations (goroutine leak risk)
3. **Dead Letter Queue:** Failed messages logged but not persisted for retry
4. **Deduplication Rollback:** Non-atomic operation creates race condition window
5. **Testing:** No unit tests (manual testing only)

Design Tradeoffs

1. **Mesa Complexity:** Agent framework adds overhead beyond simple data generation needs
2. **Fixed vs Sliding Window:** Fraud metrics use 2-hour TTL (simpler) instead of true sliding window (more precise)
3. **Python Generator Speed:** 5 TPS sufficient for development but requires scaling for production

Conclusion

Version 2 successfully addresses V1's limitations in data quality, scalability, and operational robustness. The redesign introduces production-grade patterns

including distributed caching, atomic state management, rate limiting, and type-safe communication while achieving 6,900 TPS sustained throughput with 16.9ms P99 latency.

V2 Accomplishments:

- Statistically realistic data generation suitable for ML training
- Type-safe schema preventing runtime errors
- Distributed architecture enabling horizontal scaling
- Production patterns for rate limiting and deduplication
- Validated performance under load (414K requests, 99.99% success)

V2 Deferred Work:

- ML inference layer (proven in V1, reintegration in V3)
- MLOps automation (proven in V1, migration in V3)
- Production hardening (observability, security, testing)

The phased approach of validating architecture before reintegrating ML components reduces risk and allows focused testing of each layer.
